

# HDOC Day-28

## Variable-Length 2D Arrays, Row-Major Order and 1D Array Conversion in C

### Aim

To understand variable-length arrays (VLAs), nested-loop traversal of a 2D array, column-major input, row-major output using a linear index, and conversion of a 2D array into a 1D array in row-major order.

### Theory

A two-dimensional array stores elements in rows and columns. In C, a **variable-length array (VLA)** can be declared after the row and column sizes are read at run time, for example `int a[rows][cols]`. In row-major order, all elements of one row are stored before the next row. For a linear index  $k$ , the row and column positions are obtained using  $\text{row} = k / \text{cols}$  and  $\text{column} = k \% \text{cols}$ . Nested loops are convenient for reading or copying a 2D array.

### Question 1 - Algorithm

1. Start.
2. Read rows and columns. If either value is not positive, display an error and repeat.
3. Declare the VLA `int a[rows][cols]`.
4. Read elements column by column: the outer loop iterates through columns and the inner loop iterates through rows.
5. Use one loop with linear index  $k$  from 0 to  $\text{rows} \times \text{cols} - 1$ .
6. For each  $k$ , calculate  $i = k / \text{cols}$  and  $j = k \% \text{cols}$ .
7. Print `a[i][j]`. After every `cols` elements, move to the next line.
8. Stop.

### Question 1 - Test Cases

| Test | Input                                     | Expected result                           |
|------|-------------------------------------------|-------------------------------------------|
| 1    | rows=2, cols=3; column input: 1,4,2,5,3,6 | Row-major display: 1 2 3 / 4 5 6          |
| 2    | rows=1, cols=3; 7,8,9                     | Displays one row: 7 8 9                   |
| 3    | rows=2, cols=1; 5,6                       | Displays 5 and 6 on separate rows         |
| 4    | rows=0, cols=3, then 2,3                  | Rejects invalid dimensions and asks again |
| 5    | rows=-2, cols=2, then 2,2                 | Rejects negative dimension and asks again |

## Question 1 - C Program

```
#include <stdio.h>

int main()
{
    int rows, cols, i, j, k;

    do
    {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);

        if (rows <= 0 || cols <= 0)
            printf("Invalid input! Rows and columns must be positive.\n");

    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];

    /* Column-major input */
    printf("Enter %d elements column by column:\n", rows * cols);

    for (j = 0; j < cols; j++)
    {
        for (i = 0; i < rows; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Row-major output using one loop and a linear index */
    printf("\nArray in row-major order:\n");

    for (k = 0; k < rows * cols; k++)
    {
        i = k / cols;
        j = k % cols;

        printf("%d ", a[i][j]);

        if ((k + 1) % cols == 0)
            printf("\n");
    }

    return 0;
}
```

## Question 1 - Compilation and Sample Output

Compiled successfully with **gcc -std=c99 -Wall -Wextra day28\_q1.c -o day28\_q1**.

```
Enter number of rows and columns: Enter 6 elements column by column:
a[0][0] = a[1][0] = a[0][1] = a[1][1] = a[0][2] = a[1][2] =
Array in row-major order:
1 2 3
4 5 6
```

## Question 2 - Algorithm

1. Start.
2. Read and validate positive rows and columns.
3. Declare `int a[rows][cols]` and `int b[rows*cols]`.
4. Read all elements into the 2D array using nested loops.
5. Set `k = 0`.
6. Traverse the 2D array row by row. Copy `a[i][j]` into `b[k]` and increment `k`.
7. Display the 2D array using nested loops.
8. Display the 1D array from index 0 to `rows*cols-1`.
9. Stop.

## Question 2 - Test Cases

| Test | Input                    | Expected result                                    |
|------|--------------------------|----------------------------------------------------|
| 1    | 2x3 array: 1 2 3 / 4 5 6 | 1D array: 1 2 3 4 5 6                              |
| 2    | 1x4 array: 10 20 30 40   | 1D array: 10 20 30 40                              |
| 3    | 3x1 array: 7 / 8 / 9     | 1D array: 7 8 9                                    |
| 4    | 2x2 array: -1 0 / 5 8    | 1D array: -1 0 5 8                                 |
| 5    | rows=0, cols=2, then 2,2 | Invalid dimensions rejected; valid values accepted |

## Question 2 - C Program

```
#include <stdio.h>

int main()
{
    int rows, cols, i, j, k = 0;

    do
    {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);

        if (rows <= 0 || cols <= 0)
            printf("Invalid input! Rows and columns must be positive.\n");

    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];
    int b[rows * cols];

    printf("Enter %d elements:\n", rows * cols);

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Copy 2D array to 1D array in row-major order */
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    printf("\n2D Array:\n");

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
            printf("%d\t", a[i][j]);

        printf("\n");
    }

    printf("\n1D Array in row-major order:\n");

    for (i = 0; i < rows * cols; i++)
        printf("%d ", b[i]);

    printf("\n");

    return 0;
}
```

## Question 2 - Compilation and Sample Output

Compiled successfully with **gcc -std=c99 -Wall -Wextra day28\_q2.c -o day28\_q2**.

```
Enter number of rows and columns: Enter 6 elements:
a[0][0] = a[0][1] = a[0][2] = a[1][0] = a[1][1] = a[1][2] =
2D Array:
1■2■3■
4■5■6■

1D Array in row-major order:
1 2 3 4 5 6
```

## Result

Both C programs were written, compiled and executed successfully. Program 1 validates the dimensions, accepts column-major input and prints the VLA in row-major order using a single linear index. Program 2 correctly copies a variable-length 2D array into a 1D array in row-major order and displays both arrays.